



Table of Contents

What is a Rollup Thread?	2
Accessing Retail Products	3
Get a List of Rollup Threads.....	4
API Quick Reference	7
Best Practices.....	8
Troubleshooting	8
Terms of Service	8
Document Change Log	8
Related Links	9

Adding Rollup Threads to Your Experience

Last Updated: 07/21/2020

Use the **Rollup Threads API** (<https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api>) to get related product content for your digital experience, for example to show consumers a grid wall of Nike products.

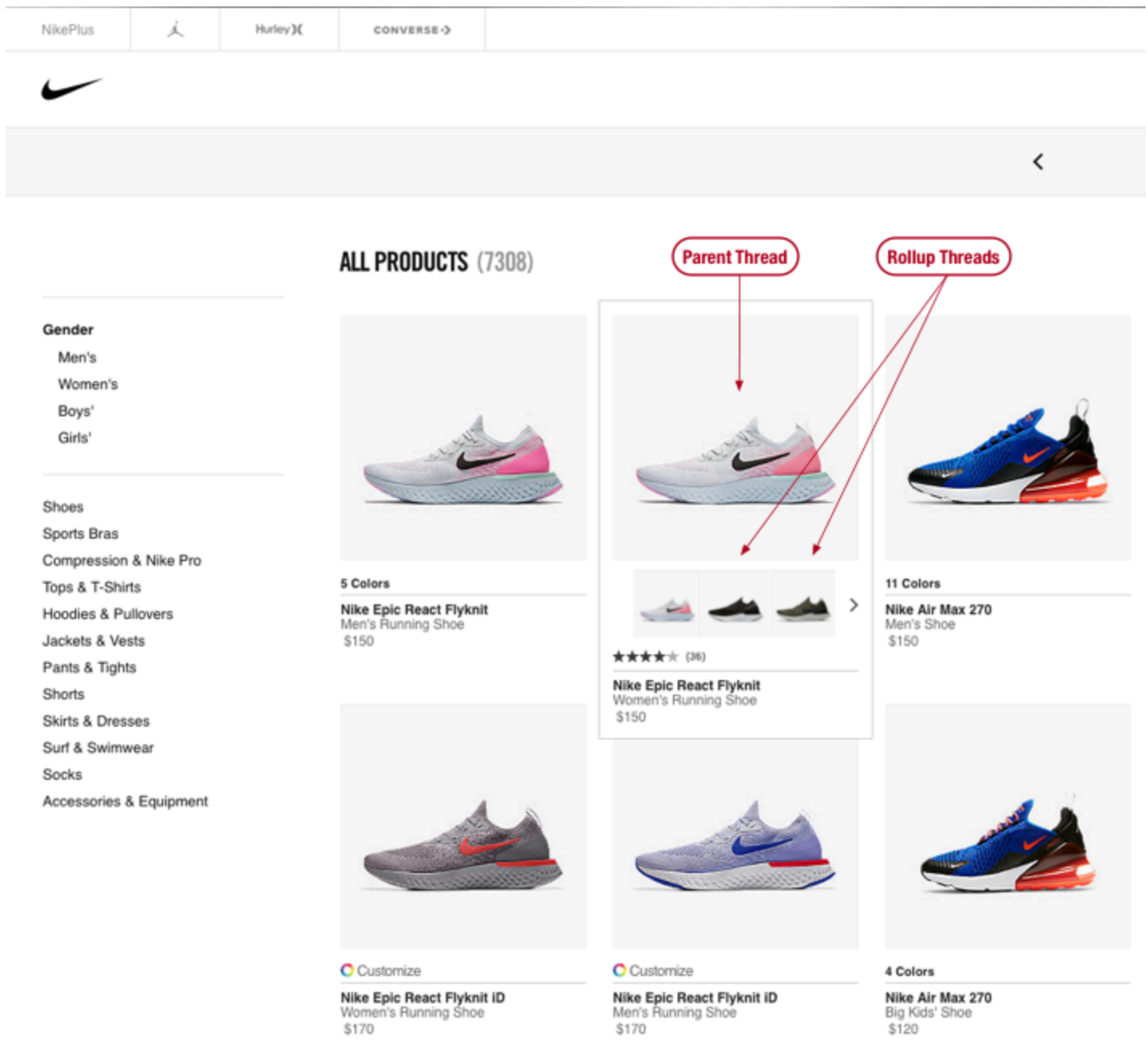
TIP: Before using this guide, first read the **Product Feeds Developer's Guide** (<https://nde-devportal-docs.niketech.com/doc/commerce/product/use-product-feeds.html>) to understand the basics about Threads.

What is a Rollup Thread?

A Thread contains content and information about a Nike product, and you can get a list of threads from the **Product Feeds API** (<https://nde-devportal-docs.niketech.com/doc/commerce/product/use-product-feeds.html>). The **Rollup Threads API** (<https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api>) takes it a step further.

A **Rollup Thread** is a Thread that is related to, and nested within, a Parent Thread. For example: a Thread containing content for a particular shoe color might have seven Rollup Threads nested within in it, one for each of the other available colors of that shoe.

Using Rollup Threads makes it simpler for you to build a product grid wall experience like this:



This is just one example of how a Parent Thread can have one or more related Rollup Threads. We'll discuss this more later in [Choosing the Rollup Threads You Need](#).

Accessing Retail Products

Both Retail and Digital products are available via **Rollup Threads** (<https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api>).

See the **Product Feeds Developer's Guide** (<https://nde-devportal-docs.niketech.com/doc/commerce/product/use-product-feeds.html>) for some important considerations for working with Retail data.

Next, we'll get started with interacting with the Rollup Threads API.

Get a List of Rollup Threads

- ✓ **Product Grid Wall: Get product threads along with related ‘rollup’ threads**
- ✓ **Get customized search results**

Getting Started

To get a list of Threads along with the associated Rollup Threads, execute a request to the **Rollup Threads List** (<https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api#rollup-threads-rollup-threads-list-get>) endpoint.

Here are some sample URIs to get you started:

Table 1: Use Cases for Rollup Threads

Use Case	Sample Query
Threads for a taxonomy ID	<code>https://api.nike.com/product_feed/rollup_threads/v2?consumerChannelId=d9a5bc42-4b9c-4976-858a-f159cf99c647&filter=marketplace(US)&filter=language(en)&filter=taxonomyIds(c2228131-f12b-4513-84cd-55ae15d6723d)</code>
Search all Nike.com products in US by “Men’s Jordan”, sorted by newest first	<code>https://api.nike.com/product_feed/rollup_threads/v2?consumerChannelId=d9a5bc42-4b9c-4976-858a-f159cf99c647&filter=language(en)&filter=marketplace(US)&searchTerms=Men’s%20Jordan&sort=effectiveStartSellDateDesc</code>

TIP: For sample responses, see the **Rollup Threads API Reference** (<https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api>).

Prerequisites

In order to use the Rollup Threads API, you will first need to:

- **Obtain a Consumer Channel ID – REQUIRED**
- **Configure Custom Search Rules – REQUIRED**

See **Consumer Channel ID & View** (<https://confluence.nike.com/pages/viewpage.action?pageId=233771813>) for instructions on the above steps.

Choosing the Parent Threads You Need

To choose only the Parent Threads that you need, append any of the **supported query parameters** (<https://developer.niketech.com/docs/projects/>)

Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api) to your request URI to get more specific results in the response.

Here are a few example query parameters.

Table 2: Example Query Parameters

Filter Criteria	Example
Women's products	<code>filter=productInfo.merchProduct.genders(WOMEN)</code>
Search keyword	<code>searchTerms=Your%20search%20terms%20here</code>
Search rules 'View'	<code>view=SEARCH_RESULTS</code>

TIPS:

- The **Product Feeds API Reference** (<https://developer.nike.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api>) is the source of truth for all supported query parameters.
- For more on Search Rules, see [Rollup by Search Rules](#)

Choosing the Rollup Threads You Need

Next, choose which Rollup Threads are sent with each Parent Thread using one of the following options:

Default Rollup

By default, the rollup is determined by the value in the **productInfo.merchProduct.productRollup.key** field in the API response. The rollup keys are created in the Nike product information system.

TIP: Generally speaking, the default rollup is by Nike style code. If you need a different rollup, use one of the below options.

Rollup by Rollup Key

Specify a particular rollup key by including the filter query parameter `rollupKey` in your request URI, for example `?filter=rollupKey(0cc4QN)`.

Rollup by Search Rules

The most powerful way to control the rollup is to configure search rules in the Apollo tool. This will **override the default rollup behavior described above, exclusively for your consumerChannelId**.

For example, you can create a rule in Apollo to exclude customized Nike ID products or gift cards from your results.

Multiple sets of search rules can be defined in **views** in Apollo and then accessed in the Rollup Threads API via

the `view` query parameter.

See [Consumer Channel ID & View \(https://confluence.nike.com/pages/viewpage.action?pageId=233771813 \)](https://confluence.nike.com/pages/viewpage.action?pageId=233771813) for instructions on how to set up views.

Consumer Channel ID Versus Channel ID

Your Consumer Channel ID is unique to your app and allows you to have custom search rules to return only the Parent and Rollup Threads that you need. But how is Consumer Channel ID related to the Channel ID you might be using with Product Feeds API?

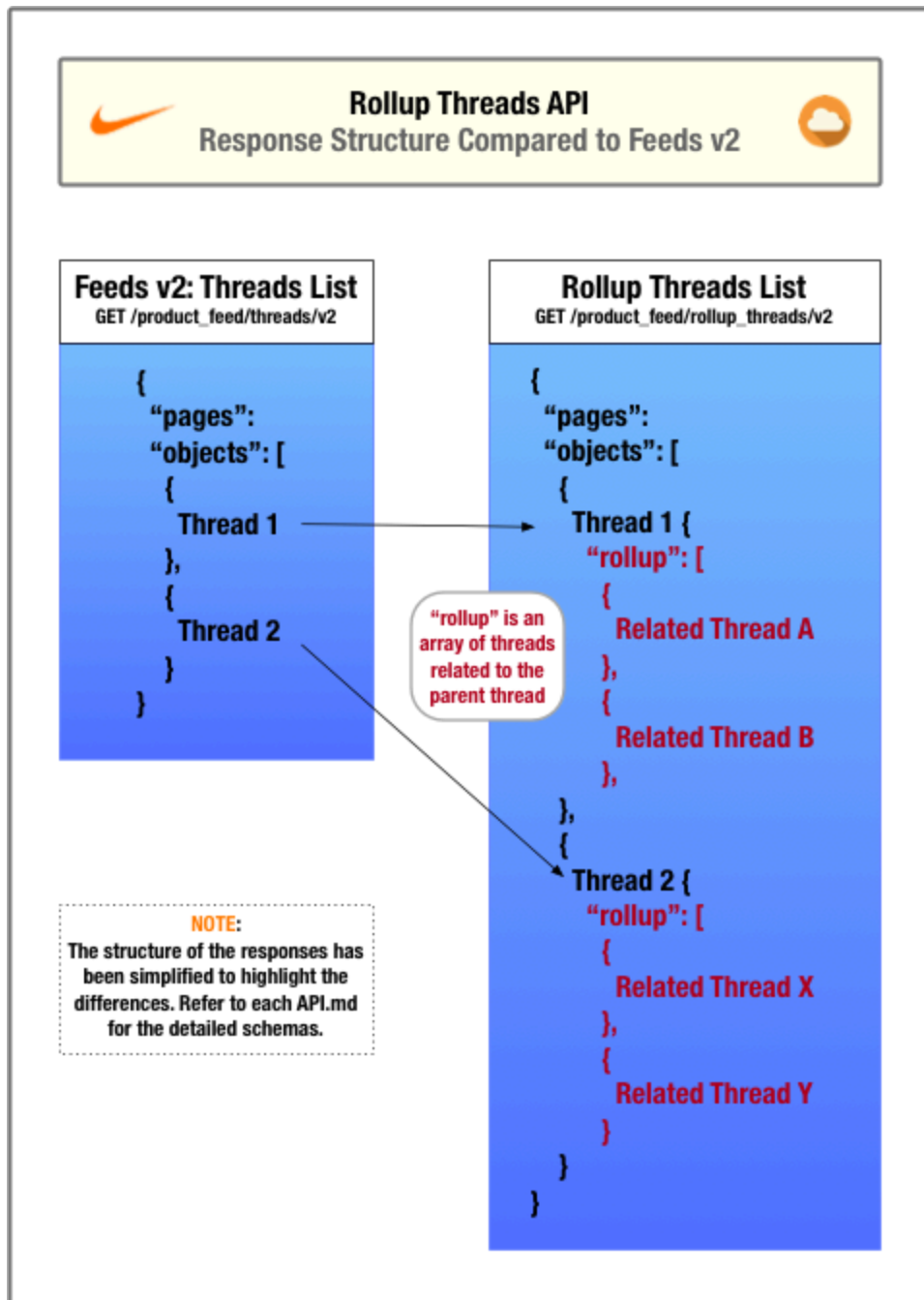
Consumer Channel ID and Channel ID are not directly related and therefore should not be used interchangeably. Both are unique IDs that help you to get only the data you need from each respective API.

When calling the Product Feed Rollup Threads API, Consumer Channel ID is required and Channel ID is not allowed.

NOTE: Threads returned by the Product Feed Rollup Threads API are pre-filtered for the Nike.com Channel ID.

I Already Use Product Feeds v2. How is This API Response Different?

The following diagram describes the how the structure of the response from the Rollup Threads API differs from Product Feeds:



API Quick Reference

Table 3: Rollup Threads Endpoints

HTTP Verb	Endpoint Name	Endpoint Description	URI Format
GET	Rollup Threads	Get a product Thread with related Threads nested within (https://developer.nike.com/docs/api/rollup-threads)	/product_feed/rollup_threads/v2{?filter,anchor,count,sort,searchTerms,r

HTTP Verb	Endpoint Name	Endpoint Description	URI Format
		ch.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api	<code>rollupCount,rollupField,consumerChannelId}</code>

Best Practices

See the Best Practices section of the [Product Feeds Developer's Guide \(https://nde-devportal-docs.niketech.com/doc/commerce/product/use-product-feeds.html#best-practices \)](https://nde-devportal-docs.niketech.com/doc/commerce/product/use-product-feeds.html#best-practices).

Troubleshooting

I'm getting a 200 response, but the Thread data and/or Rollup Threads are not as expected

- Check your Smart Search rules configuration in the Apollo application to ensure that the rules are correct.
- Check the rollup key & type from Prodigy for the Parent Thread is as expected.
- Reach out to Product Feeds team on Slack for assistance: [#nde-product-feeds \(https://nikedigital.slack.com/messages/CAPF62A66 \)](https://nikedigital.slack.com/messages/CAPF62A66)

TIP: See the Troubleshooting section of the [Product Feeds Developer's Guide \(https://nde-devportal-docs.niketech.com/doc/commerce/product/use-product-feeds.html#troubleshooting \)](https://nde-devportal-docs.niketech.com/doc/commerce/product/use-product-feeds.html#troubleshooting) for more general troubleshooting information.

Terms of Service

It is recommended that you send a caller ID header in every request to this API to help troubleshoot unexpected responses. See the Registration section of [Using Nike APIs \(https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html#registration \)](https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html#registration) on how to create and register your caller ID.

Authentication

No authentication or authorization is required to use this API.

Document Change Log

Summary	Date
Initial publish	05/17/2018
Updated how to obtain a consumerChannelId	07/19/2018
Referenced new Retail data	07/06/2020

Related Links

- [Using Nike APIs \(https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html \)](https://nde-devportal-docs.niketech.com/doc/getting-started/using-nike-apis.html)
- [Glossary \(https://nde-devportal-docs.niketech.com/doc/commerce/reference/glossary.html \)](https://nde-devportal-docs.niketech.com/doc/commerce/reference/glossary.html)